Національний технічний університет України
«Київський політехнічний інститут ім. Ігоря Сікорського»
Інститут прикладного системного аналізу
Кафедра системного проектування

Лабораторна робота №1

З дисципліни «Операційні системи»

На тему: «**Основи багатопотокового програмування.**
Синхронізація та взаємодія процесів та потоків»

Виконав:
студент 2 курсу
групи ДА-11
Сидько Нікіта

Київ – 2023

Завдання:

Варіант	«виробники/ споживачі»	«обідаючи філософи»	«читачі/ письменники»	«сплячий цирульник»
22	семафори (WIN32)	взаємовиключення (UNIX)	семафори (UNIX)	критичні секції (WIN32)

1.1) Розробити рішення класичної задачі синхронізації – «виробників/споживачів» за допомогою семафорів у середовищі win32

Проблема виробника та споживача (проблема обмеженого буфера).Два процеси спільно використовують буфер обмеженого розміру. Один із них, виробник, поміщає дані до цього буфера, а інший, споживач, зчитує їх звідти. Завдання може бути узагальнена на випадок N виробників і M споживачів.

Лістинг коду

My_queue.h

```
#pragma once
#include <iostream>
using namespace std;

class Queue
{
	struct Node;

private:
	int size;
	Node* head;
	Node* tail;

public:
	Queue();
	~Queue();

	void push(int a);
	int pop();
	int get(int a);
	int get_size();
	void print_queue();
};
```

My_queue.cpp

```
#include "my_queue.h"


struct Queue::Node
{
	int data;
	Node* next;
	Node(int data, Node* next)
	{
		this->data = data;
		this->next = next;
	}
};


Queue::Queue()
```

```
{
	head = nullptr;
	tail = nullptr;
	size = 0;
}

Queue:: ~Queue()
{
	if (head == nullptr)
	{
		return;
	}
	Node* next = head->next;
	while (next != nullptr)
	{
		delete head;
		head = next;
		next = head->next;
	}
	delete head;
}

void Queue::push (int data)
{
	if (tail == nullptr)
	{
		tail = new Node(data, nullptr);
		head = tail;
	}
	else
	{
		tail->next = new Node(data, nullptr);
		tail = tail->next;
	}
	size++;
}

void Queue :: print_queue()
{
	if (head == nullptr)
	{
		cout << "Queue is empty.";
		return;
	}
	cout << "Queue: ";

	Node* temp = head;

	for (int i = 0; i < size; i++)
	{
		cout << temp->data << " ";
		temp = temp->next;
	}

}

int Queue::pop()
{
	if (head == nullptr)
	{
		cout << "Queue is empty." << endl;
		return -1;
	}

	int ret_data = head->data;
	Node* new_head = head->next;
	delete head;
	head = new_head;
	if (head == nullptr)
```

```
	{
		tail = head;
	}
	size--;
	return ret_data;
}

int Queue::get(int n)
{
	if (n < 0 || n > size)
	{
		cout << "Cannot find element under this index." << endl;
	}
	if (head == nullptr)
	{
		cout << "Queue is empty." << endl;
	}

	Node* temp = head;
	for (int i = 0; i < n; i++)
	{
		temp = temp->next;
	}
	return temp->data;
}

int Queue::get_size()
{
	return size;
}
```

Main.cpp

```
#include <iostream>
#include <windows.h>
#include <vector>
#include "my_queue.h"

using namespace std;

HANDLE producer;
HANDLE consumer;
HANDLE empty_queue;
HANDLE full_queue;
HANDLE block_queue;

DWORD WINAPI ThreadProdProc(LPVOID param)
{
	Queue* queue = static_cast<Queue*>(param);

	int item = 0;
	int i = 0;
	srand(GetCurrentThreadId());

	while (++i < 3)
	{
		WaitForSingleObject(producer, INFINITE);
		WaitForSingleObject(empty_queue, INFINITE);
		WaitForSingleObject(block_queue, INFINITE);
		item = rand();
		queue->print_queue();
		cout << "\nQueue size = " << queue->get_size() << "\nProducer " <<
GetCurrentThreadId() << " is pushing " << item << endl << endl;

		if (queue->get_size() == 1)
		{
			queue->push(item);
			ReleaseSemaphore(block_queue, 1, NULL);
```

```
		}
		else
		{
			ReleaseSemaphore(block_queue, 1, NULL);
			queue->push(item);
		}

		Sleep(500);


		ReleaseSemaphore(full_queue, 1, NULL);
		ReleaseSemaphore(producer, 1, NULL);
	}
	return true;
}


DWORD WINAPI ThreadConsProc(LPVOID param)
{
	Queue* queue = static_cast<Queue*>(param);
	int item = 0;
	int i = 0;
	srand(GetCurrentThreadId());

	while (++i < 3)
	{
		WaitForSingleObject(consumer, INFINITE);
		WaitForSingleObject(full_queue, INFINITE);
		WaitForSingleObject(block_queue, INFINITE);
		item = queue->get(0);
		queue->print_queue();
		cout << "\nQueue size = " << queue->get_size() << "\nConsumer " <<
GetCurrentThreadId() << " is popping " << item << endl << endl;

		if (queue->get_size() == 1)
		{
			queue->pop();
			ReleaseSemaphore(block_queue, 1, NULL);
		}
		else
		{
			ReleaseSemaphore(block_queue, 1, NULL);
			queue->pop();
		}
		Sleep(1000);
		ReleaseSemaphore(empty_queue, 1, NULL);
		ReleaseSemaphore(consumer, 1, NULL);
	}
	return true;
}

int main()
{
	int queue_size, producer_num, consumer_num;
	Queue queue;
	cout << "Enter max size of queue: ";
	cin >> queue_size;
	cout << "Enter the number of producers: ";
	cin >> producer_num;
	cout << "Enter the number of consumers: ";
	cin >> consumer_num;
	cout << endl;

	vector<HANDLE> producer_threads(producer_num);
	vector<HANDLE> consumer_threads(consumer_num);

	producer = CreateSemaphore(NULL, 1, 1, NULL);
	consumer = CreateSemaphore(NULL, 1, 1, NULL);
	empty_queue = CreateSemaphore(NULL, queue_size, queue_size, NULL);
	full_queue = CreateSemaphore(NULL, 0, queue_size, NULL);
```

```
	block_queue = CreateSemaphore(NULL, 1, 1, NULL);

	for (int i = 0; i < producer_num; i++)
	{
		if ( (producer_threads[i] = CreateThread(NULL, 0, ThreadProdProc, &queue, 0,
NULL)) == NULL )
		{
			cout << "CreateThread error." << endl;
		}
	}

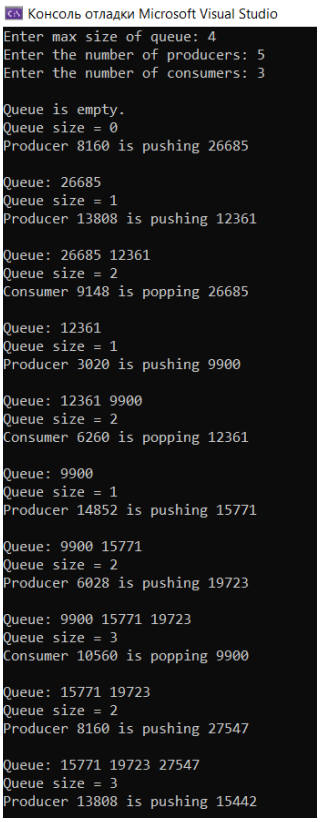
	for (int i = 0; i < consumer_num; i++)
	{
		if ((consumer_threads[i] = CreateThread(NULL, 0, ThreadConsProc, &queue, 0,
NULL)) == NULL)
		{
			cout << "CreateThread error." << endl;
		}
	}

	for (int i = 0; i < producer_num; i++)
	{
		WaitForSingleObject(producer_threads[i], INFINITE);
	}
	for (int i = 0; i < consumer_num; i++)
	{
		WaitForSingleObject(consumer_threads[i], INFINITE);
	}

	CloseHandle(producer);
	CloseHandle(consumer);
	CloseHandle(empty_queue);
	CloseHandle(full_queue);
	CloseHandle(block_queue);

	return 0;
}
```

Результат роботи коду :



1.2) Розробити рішення класичної задачі синхронізації – «обідаючі філософи» за допомогою взаємовиключень UNIX

Умови завдання: Є N філософів, які сидять за круглим столом. Перед кожним із них знаходиться тарілка зі спагеті. Спагеті настільки слизькі, що кожному філософу необхідно дві виделки, щоб з ними впоратися. Між кожними двома тарілками лежить одна виделка. Життя філософа складається з періодів поглинання їжі і роздумів, що чергуються (інші процеси життєдіяльності в даному випадку несуттєві). Коли філософ голодний, він намагається отримати дві виделки, ліву та праву, у будь-якому порядку. Якщо йому вдалося отримати дві виделки, він деякий час їсть, потім кладе виделки і продовжує роздуми. Ця проблема корисна для моделювання процесів, що змагаються за монопольний доступ до обмеженої кількості ресурсів (наприклад, пристроїв введення-виведення).

Лістинг коду

```
#include <iostream>
#include <pthread.h>
#include <unistd.h>

using namespace std;
const int NUM = 5;
const int MEALS_NUM = 1;

pthread_mutex_t forks[NUM];
pthread_mutex_t CoutMutex;

void* Philosopher(void* data)
{
    int count = 0;
    int philosopher_number = *(int*)data;
    int right_fork = (philosopher_number + 1) % NUM;
    int left_fork = (philosopher_number) % NUM;
    sleep(rand() % 5);

    while (count < MEALS_NUM)
    {
        if (!pthread_mutex_trylock(&forks[left_fork]))
        {
            if (!pthread_mutex_trylock(&forks[right_fork]))
            {
                pthread_mutex_lock(&CoutMutex);
                cout << "Philosopher " << philosopher_number + 1 << " start eating" <<
endl;
                pthread_mutex_unlock(&CoutMutex);
                sleep(rand() % 5);
                pthread_mutex_lock(&CoutMutex);
                count++;
                cout << "Philosopher " << philosopher_number + 1 << " stop eating (" <<
count << "/" << MEALS_NUM << ")" << endl;
                pthread_mutex_unlock(&CoutMutex);

                pthread_mutex_unlock(&forks[right_fork]);
            }
            pthread_mutex_unlock(&forks[left_fork]);
        }
        sleep(rand() % 10);
    }
}

int main()
{
    srand(time(NULL));
```

```
    for (int i = 0; i < NUM; i++)
    {
        pthread_mutex_init(&forks[i], NULL);
    }
    pthread_mutex_init(&CoutMutex, NULL);

    pthread_t philosophers[NUM];
    for (int i = 0; i < NUM; i++)
    {
        int* temp = new int(i);
        pthread_create(&philosophers[i], NULL, &Philosopher, temp);
    }
    for (int i = 0; i < NUM; i++)
    {
        pthread_join(philosophers[i], NULL);
    }

    for (int i = 0; i < NUM; i++)
    {
        pthread_mutex_destroy(&forks[i]);
    }
    pthread_mutex_destroy(&CoutMutex);
    return 0;
}
```

Результат роботи коду :

```
/home/nikita/CLionProjects/OS_lab_1/cmake-build-debug/OS_lab_1
Philosopher 2 start eating
Philosopher 5 start eating
Philosopher 2 stop eating (1/1)
Philosopher 5 stop eating (1/1)
Philosopher 1 start eating
Philosopher 1 stop eating (1/1)
Philosopher 4 start eating
Philosopher 4 stop eating (1/1)
Philosopher 3 start eating
Philosopher 3 stop eating (1/1)

Process finished with exit code 0
```

1.3) Розробити рішення класичної задачі синхронізації – «читачі/письменники» за допомогою семафорів (UNIX)

Умова завдання: необхідно забезпечити монопольний доступ до бази даних процесу, що виконує запис даних, та множинний доступ до процесів, що виконують читання даних. Завдання може бути сформульована з наданням пріоритету як процесу-письменнику, так і процесам-читачам.

Лістинг коду

```
#include <iostream>
#include <pthread.h>
#include <unistd.h>
#include <semaphore.h>

using namespace std;

#define READERS_NUM 4
#define WRITERS_NUM 2
#define READERS_VISIT 1
#define  WRITERS_VISIT 1
```

```
sem_t ReadWriteSem;
sem_t VariablesSem;
int r_count = 0;
int w_count = 0;
bool flag = false;

void* Readers(void* data)
{
    int num = *(int*)data;
    sleep(rand() % 5);
    int counter = 0;

    while (counter < READERS_VISIT)
    {

        if (flag)
        {
            sleep(rand() % 10);
            continue;
        }


        sem_wait(&VariablesSem);
        if (r_count == 0)
        {
            sem_post(&VariablesSem);
            sem_wait(&ReadWriteSem);
            sem_wait(&VariablesSem);
        }

        r_count++;
        cout << "Reader " << num << " start (" << counter + 1 << " time)." << " Total: " <<
r_count << endl;
        sem_post(&VariablesSem);
        sleep(rand() % 10);


        sem_wait(&VariablesSem);
        r_count--;
        cout << "Reader " << num << " stop (" << counter + 1 << " time)." << " Total: " <<
r_count << endl;
        sem_post(&VariablesSem);


        if (r_count == 0)
        {
            sem_post(&ReadWriteSem);
        }
        counter++;

        sleep(rand() % 2);
    }
    return 0;
}

void* Writers(void* data)
{
    int num = *(int*)data;
    int counter = 0;

    while (counter < WRITERS_VISIT)
    {
        sleep(rand() % 10);
        flag = true;
        sem_wait(&ReadWriteSem);
        flag = false;
        w_count++;
```

```
        cout << "Writer " << num << " start (" << counter + 1 << " time)." << " Total: " <<
w_count << endl;
        sleep(2);
        w_count--;
        cout << "Writer " << num << " stop (" << counter + 1 << " time)." << " Total: " <<
w_count << endl;
        sem_post(&ReadWriteSem);

        counter++;
    }
    return 0;
}

int main()
{
    srand(time(NULL));

    sem_init(&VariablesSem, 0, 1);
    sem_init(&ReadWriteSem, 0, 1);

    pthread_t ReaderThreads[READERS_NUM];
    pthread_t WriterThreads[WRITERS_NUM];

    for (int i = 0; i < READERS_NUM; i++)
    {
        int* temp = new int(i);

        pthread_create(&ReaderThreads[i], NULL, Readers, temp);
    }
    for (int i = 0; i < WRITERS_NUM; i++)
    {
        int* temp = new int(i);

        pthread_create(&WriterThreads[i], NULL, Writers, temp);
    }



    for (int i = 0; i < READERS_NUM; i++)
    {
        pthread_join(ReaderThreads[i], NULL);
    }
    for (int i = 0; i < WRITERS_NUM; i++)
    {
        pthread_join(WriterThreads[i], NULL);
    }

    sem_destroy(&ReadWriteSem);
    sem_destroy(&VariablesSem);

    return 0;
}
```

Результат роботи коду :

```
/home/nikita/CLionProjects/OS_3/cmake-build-debug/OS_3
Reader 0 start (1 time). Total: 1
Reader 0 stop (1 time). Total: 0
Reader 1 start (1 time). Total: 1
Reader 3 start (1 time). Total: 2
Reader 3 stop (1 time). Total: 1
Reader 1 stop (1 time). Total: 0
Writer 0 start (1 time). Total: 1
Writer 0 stop (1 time). Total: 0
Writer 1 start (1 time). Total: 1
Writer 1 stop (1 time). Total: 0
Reader 2 start (1 time). Total: 1
```

1.4) Розробити рішення класичної задачі синхронізації – «сплячий цирюльник» за допомогою критичних секцій у середовищі win32

Умова завдання: є один процес-«цирульник», його крісло і N стільців для процесів-«клієнтів». Якщо «клієнтів немає», «цирульник» спить у своєму кріслі. Якщо в перукарню приходить «клієнт», він має розбудити «цирульника». Якщо клієнт приходить і бачить, що цирульник зайнятий, він або займає один із стільців (якщо є місце), або йде (якщо місця немає). Необхідно запрограмувати цирульника і клієнтів так, щоб уникнути стану змагання.

Лістинг коду

My_queue.h

```
#pragma once
#include <stdexcept>
#include <iostream>
using namespace std;

class Queue
{
private:

	struct Node;
	Node* head;
	Node* tail;

public:
	Queue();
	~Queue();

	void push(int key);

	int pop();

	int getSize();

	void printQueue();

};
```

My_queue.cpp

```
#include "my_queue.h"

using namespace std;


struct Queue::Node
{
	int key;
	Node* next;
	Node(int key, Node* next)
	{
		this->key = key;
		this->next = next;
	}
};

Queue::Queue()
{
```

```
	head = nullptr;
	tail = nullptr;
}

Queue :: ~Queue()
{
	Node* current = head;
	while (current)
	{
		Node* next = current->next;
		delete current;
		current = next;
	}
}


void Queue::push(int key)
{
	Node* newNode = new Node(key, nullptr);

	if (tail == nullptr)
	{
		head = newNode;
		tail = newNode;
	}
	else
	{
		tail->next = newNode;
		tail = newNode;
	}
}

int Queue::pop()
{
	if (head == nullptr)
	{
		throw length_error("Queue is empty.");
	}

	Node* temp = head;
	int val = temp->key;

	head = head->next;
	if (head == nullptr)
	{
		tail = nullptr;
	}

	delete temp;
	return val;
}

int Queue:: getSize()
{
	int size = 0;
	Node* current = head;
	while (current)
	{
		size++;
		current = current->next;
	}
	return size;

}

void Queue::printQueue()
{
	cout << "Queue: ";
	Node* current = head;
```

```
	while (current)
	{
		cout << current->key << " ";
		current = current->next;
	}
	cout << endl;
}
```

Main.cpp

```
#include <windows.h>
#include <iostream>
#include <stdexcept>
#include "my_queue.h"

using namespace std;

#define VISITORS_NUM 8
#define CHAIRS_NUM 2

CRITICAL_SECTION visitors_queue;
CRITICAL_SECTION output;
CRITICAL_SECTION barber;
CRITICAL_SECTION barber1;

Queue* visitors;

DWORD WINAPI functionBarber(LPVOID arg)
{
	while (true)
	{
		EnterCriticalSection(&visitors_queue);

		if (visitors->getSize() == 0)
		{
			LeaveCriticalSection(&visitors_queue);
			EnterCriticalSection(&output);
			cout << "Barber is sleeping" << endl;
			LeaveCriticalSection(&output);

			Sleep(1000);
			EnterCriticalSection(&barber1);
		}
		else
		{
			int clientId = visitors->pop();
			LeaveCriticalSection(&visitors_queue);

			Sleep(2000 + rand() % 2000);

			EnterCriticalSection(&output);
			cout << "Barber done work and client " << clientId << " go away" <<
endl;
			LeaveCriticalSection(&output);
			LeaveCriticalSection(&barber);
		}
	}
	return true;
}

DWORD WINAPI functionVisit(LPVOID arg)
{
	int clientId = *static_cast<int*>(arg) + 1;

	EnterCriticalSection(&visitors_queue);

	if ((CHAIRS_NUM - visitors->getSize()) > 0)
	{
		EnterCriticalSection(&output);
```

```
            visitors->push(clientId);
            cout << "Client " << clientId << " added to queue, queue size: " << visitors-
>getSize() << endl;
            visitors->printQueue();
            LeaveCriticalSection(&visitors_queue);
            LeaveCriticalSection(&output);

            if (visitors->getSize() == 1)
            {
                  LeaveCriticalSection(&barber1);
            }
            EnterCriticalSection(&barber);

            EnterCriticalSection(&output);
            cout << "Barber is working: " << clientId << endl;
            LeaveCriticalSection(&output);
      }
      else
      {
            LeaveCriticalSection(&visitors_queue);

            EnterCriticalSection(&output);
            cout << "No chairs, client " << clientId << " go away" << endl;
            LeaveCriticalSection(&output);
      }

      return true;
}

int main()
{
      DWORD threadId;

      HANDLE barberThread;
      HANDLE visitorsThreads[VISITORS_NUM];
      int visitorsIds[VISITORS_NUM];

      visitors = new Queue();

      InitializeCriticalSection(&output);
      InitializeCriticalSection(&visitors_queue);
      InitializeCriticalSection(&barber);
      InitializeCriticalSection(&barber1);

      EnterCriticalSection(&barber1);

      barberThread = CreateThread(NULL, 0, &functionBarber, NULL, 0, &threadId);

      for (int i = 0; i < VISITORS_NUM; ++i)
      {
            visitorsIds[i] = i;
            Sleep(1000 + rand() % 500);
            visitorsThreads[i] = CreateThread(NULL, 0, &functionVisit, &visitorsIds[i], 0,
&threadId);
      }
      WaitForSingleObject(barberThread, INFINITE);
      WaitForMultipleObjects(VISITORS_NUM, visitorsThreads, true, INFINITE);

      CloseHandle(barberThread);

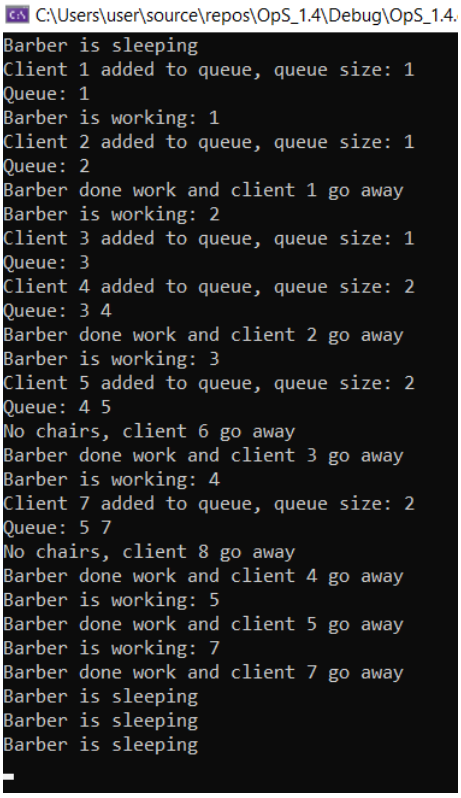
      for (int j = 0; j < VISITORS_NUM; ++j)
      {
            CloseHandle(visitorsThreads[j]);
      }

      DeleteCriticalSection(&output);
      DeleteCriticalSection(&visitors_queue);
      DeleteCriticalSection(&barber);
```

```
        delete visitors;

        return 0;
}
```

Результат роботи коду:



Висновок.

У ході виконання лабораторної роботи №1 було вивчено об'єкти, функції та системні виклики ОС сімейства UNIX (FreeBSD, Linux) та WIN32 API, а також стандартні бібліотеки C++. Це дало мені практичні навички розробки багатопотокових програм з використанням різних методів синхронізації. У моїй роботі було реалізовано вирішення класичних задач синхронізації, таких як "обідаючі філософи", "виробники/споживачі", "читачі/письменники" і "сплячий цирульник". Для цього були використані критичні секції, семафори та взаємовиключення . Також було використано чергу на основі зв’язного списку, як об'єкт синхронізації, коли це було необхідно та логічно.